kafka-advanced

Production-Grade Apache Kafka Microservices

Complete Technical Reference — HLD · LLD · Architecture · Configuration

Author: Mohammad Mateen | Date: 2026-10-17

Version 1.0 | Java 17 · Spring Boot 3.3.4 · Apache Kafka 7.6.0

Phase 1 — Practical	Phase 2 — Intermediate	Phase 3 — Production
Idempotent Producer Partitioning by Key Retry + DLT Global Error Handler	Kafka Streams Consumer Lag Monitoring Schema Registry + Avro	Kafka Transactions Kafka Connect SSL/SASL Security

Table of Contents

1. Project Overview
2. System Architecture (HLD)
3. Services & Technology Stack
4. Kafka Topics Reference
5. Phase 1 — Practical Kafka (LLD)
    5.1 Idempotent Producer — order-service
    5.2 Partitioning by Key
    5.3 Retry + Dead Letter Topic — payment-service
    5.4 Global Error Handler — shipping-service
    5.5 DLT Consumer — dlt-handler-service
6. Phase 2 — Intermediate Kafka (LLD) ★ Advanced
    6.1 Kafka Streams — audit-service
    6.2 Consumer Lag Monitoring
    6.3 Schema Registry + Avro — avro-service
7. Phase 3 — Production Kafka (LLD) ★ Advanced
    7.1 Kafka Transactions
    7.2 Kafka Connect — JDBC Sink
    7.3 SSL/SASL Security
8. Infrastructure & Docker Compose
9. Port Reference
10. How to Run — Step by Step
11. Test Scenarios
12. Configuration Reference

1. Project Overview

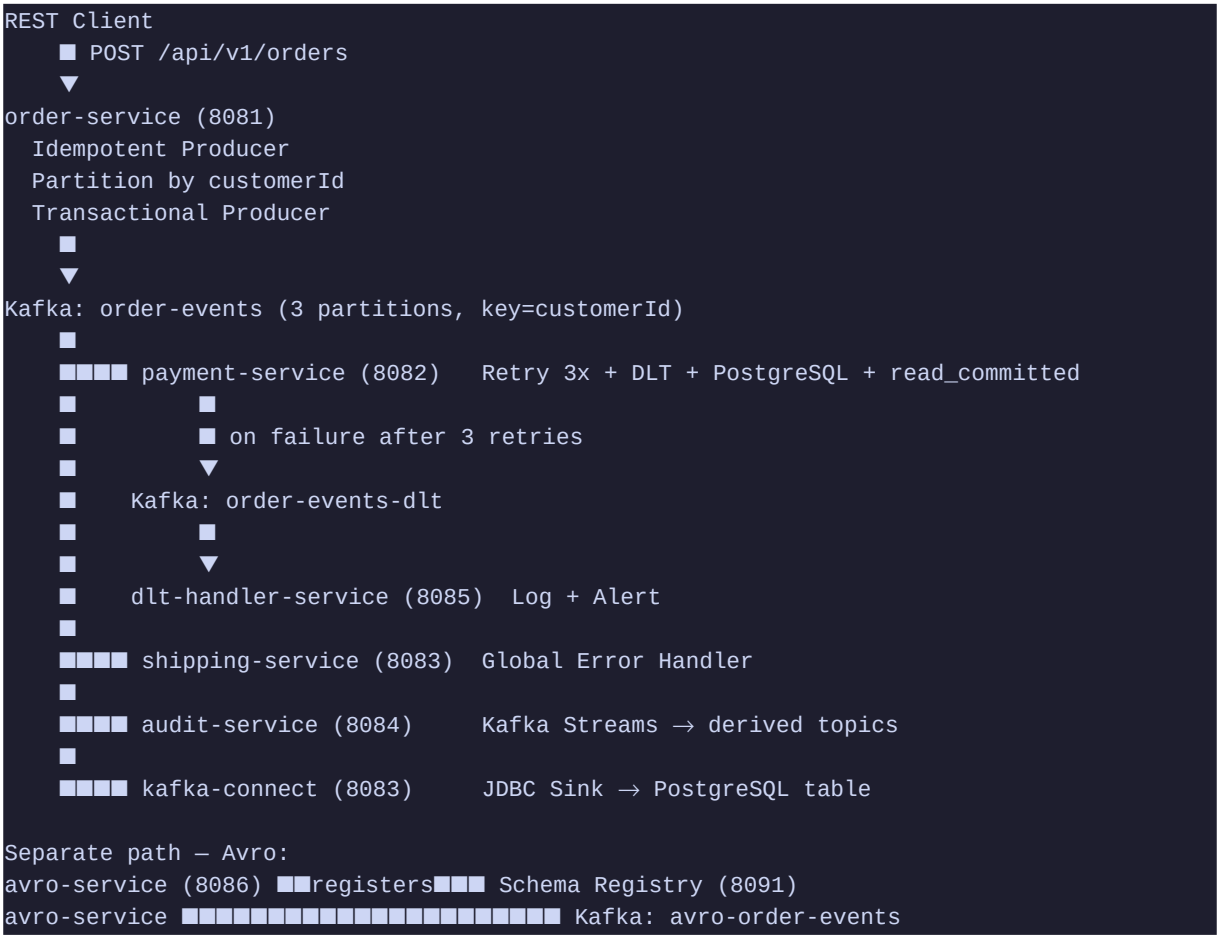
kafka-advanced is a production-grade, event-driven microservices system built with Spring Boot 3.3.4 and Apache Kafka 7.6.0. It demonstrates every major Kafka concept needed in real-world enterprise systems — from basic consumer groups to advanced schema governance, stream processing, and exactly-once delivery guarantees.

The system models an e-commerce order processing pipeline where customer orders flow through payment, shipping, audit, and analytics microservices. Each service showcases a distinct Kafka pattern with full working code, configuration, and documentation.

Category	Technology	Version
Language	Java	17
Framework	Spring Boot	3.3.4
Message Broker	Apache Kafka (Confluent)	7.6.0
Schema Registry	Confluent Schema Registry	7.6.0
Serialization	Apache Avro	1.11.3
Database	PostgreSQL	15
Observability	ELK Stack (Elastic)	8.15.3
Tracing	Zipkin	3.4
Build Tool	Apache Maven	3.x
Containerization	Docker Compose	3.8

2. System Architecture — High-Level Design (HLD)

The system follows an event-driven architecture. The order-service acts as the sole Kafka producer. All other services are independent consumers, each implementing a different Kafka pattern. Services do not call each other directly — all communication flows through Kafka topics.



Data Flow — Order Placed

Step	Action
1	Client sends POST /api/v1/orders with OrderDTO
2	order-service sets orderId=UUID, status=PENDING, wraps in OrderEventDTO
3	Idempotent producer sends to order-events — Key=customerId → same partition always
4	THREE consumers receive same event independently: payment, shipping, audit
5	payment-service processes payment → saves to PostgreSQL; on failure retries 1s→2s→4s
6	After 3 retries: message moves to order-events-dlt → dlt-handler-service
7	shipping-service schedules shipment; global error handler catches exceptions
8	audit-service Kafka Streams: counts orders per customer, aggregates revenue per product
9	kafka-connect: reads order-events, upserts each message to order_events_sink PostgreSQL table

3. Services & Technology Stack

Service	Port	Concepts Covered	Phase
order-service	8081	Idempotent Producer, Partition by key, Transactions	1 + 3
payment-service	8082	Retry 3x, DLT, PostgreSQL, Consumer Lag, read_committed	1 + 2
shipping-service	8083	Global Error Handler, Consumer Lag	1 + 2
audit-service	8084	Kafka Streams, KStream, KTable, State Stores	2
dlt-handler-service	8085	DLT Consumer, Consumer Lag Monitoring	1 + 2
avro-service	8086	Schema Registry, Avro, SpecificRecord, avro-maven-plugin	2
kafka-connect	8083*	JDBC Sink Connector, auto.create, Flatten transform	3
security-config	—	SASL/PLAIN, JAAS, docker-compose-sasl	3
base-domain	—	Shared DTOs, Enums (OrderDTO, OrderEventDTO, etc.)	1

■ *Kafka Connect REST API runs in Docker on port 8083 — separate from shipping-service.

4. Kafka Topics Reference

Topic	Partitions	Key	Producer	Consumers
order-events	3	customerId	order-service	payment, shipping, audit, connect
order-confirmed-events	3	customerId	order-service (TX)	—
order-events-retry-0	1	—	Spring auto	payment-service
order-events-retry-1	1	—	Spring auto	payment-service
order-events-retry-2	1	—	Spring auto	payment-service
order-events-dlt	1	—	Spring auto	dlt-handler-service
avro-order-events	1	customerId	avro-service	avro-service
order-count-per-customer	1	customerId	audit (Streams)	—
revenue-per-product	1	productName	audit (Streams)	—

5. Phase 1 — Practical Kafka (Low-Level Design)

5.1 Idempotent Producer — order-service

An idempotent producer guarantees that each message is written to Kafka exactly once, even when the producer retries after a network error. Kafka assigns a sequence number to each message. If the same sequence number arrives twice, Kafka discards the duplicate.

```
# application.yml — order-service
spring.kafka.producer.properties:
  enable.idempotence: true
  acks: all
  retries: 3
  max.in.flight.requests.per.connection: 5
```

■ Rule: max.in.flight.requests must be ≤ 5 when idempotence is enabled. acks must be 'all' (all replicas confirm before ACK).

5.2 Partitioning by Key (customerId)

All orders from the same customer are routed to the same Kafka partition by using customerId as the message key. Kafka uses murmur2(key) % numPartitions to determine the partition. This guarantees message ordering per customer.

```
// OrderKafkaProducer.java
// Key = customerId → same partition per customer
kafkaTemplate.send(orderEventsTopic,
    orderEventDTO.getOrder().getCustomerId(),  // KEY
    orderEventDTO);                            // VALUE
```

5.3 Retry + Dead Letter Topic — payment-service

@RetryableTopic creates separate retry topics automatically. After attempts-1 retries, the message is moved to a DLT topic. The @DltHandler method handles it for manual intervention. Retry topics use exponential backoff.

```
@RetryableTopic(
    attempts = "4",                      // 1 original + 3 retries
    backoff = @Backoff(delay = 1000, multiplier = 2.0),
    topicSuffixingStrategy = SUFFIX_WITH_INDEX_VALUE,
    dltTopicSuffix = "-dlt"
)
@KafkaListener(topics = "order-events", groupId = "payment-service-group")
public void consume(OrderEventDTO event) { ... }

@DltHandler
public void handleDlt(OrderEventDTO event) {
    // Called after all retries exhausted
    // Action: alert ops, create ticket, notify customer
}
```

Topic	Trigger	Wait
order-events	Original attempt fails	—
order-events-retry-0	Attempt 1 fails	1 second
order-events-retry-1	Attempt 2 fails	2 seconds
order-events-retry-2	Attempt 3 fails	4 seconds

order-events-dlt	Attempt 4 fails	@DltHandler called

5.4 Global Error Handler — shipping-service

A single DefaultErrorHandler bean applies to ALL @KafkaListener methods in the service. Non-retryable exceptions (IllegalArgumentException, NullPointerException) skip retries and fail immediately — no point retrying bad data.

```
@Bean
public DefaultErrorHandler errorHandler() {
    FixedBackOff backOff = new FixedBackOff(1000L, 3L); // 1s, 3 attempts
    DefaultErrorHandler handler = new DefaultErrorHandler(
        (record, ex) -> LOGGER.error("All retries exhausted"), backOff);
    handler.addNotRetryableExceptions(
        IllegalArgumentException.class,
        NullPointerException.class);
    return handler;
}
```

5.5 DLT Consumer — dlt-handler-service

A dedicated service consuming only from order-events-dlt. This separation means DLT handling logic is isolated from business logic. In production, this service would send alerts, create support tickets, and trigger manual reprocessing workflows.

6. Phase 2 — Intermediate Kafka (Low-Level Design)

★ ADVANCED TOPIC: Phase 2 covers advanced Kafka features not commonly found in beginner projects.

6.1 Kafka Streams — audit-service

★ ADVANCED TOPIC: Kafka Streams: client-side library for stateful stream processing — no separate cluster needed.

audit-service uses the Kafka Streams DSL to read the order-events topic and produce two derived topics in real-time: order count per customer and revenue per product. Aggregations are backed by persistent state stores (RocksDB) that survive service restarts.

```
// Stream 1: Count orders per customer
KTable countTable = orderStream
    .groupByKey(Grouped.with(stringSerde, stringSerde))
    .count(Materialized.as("order-count-store"));

// Stream 2: Revenue per product
orderStream
    .mapValues(json -> extractProductName(json) + ":" + extractPrice(json))
    .selectKey((customerId, productPrice) -> productPrice.split(":")[0])
    .groupByKey()
    .aggregate(
        () -> "0",  // initializer
        (productName, productPrice, currentRevenue) ->
            new BigDecimal(currentRevenue).add(new BigDecimal(price)).toString(),
        Materialized.as("revenue-store")
    );
```

Concept	Description
KStream	Unbounded, append-only stream — like a river. Every message is an event.
KTable	Latest value per key — like a database table with live updates.
groupByKey()	Group records by their existing key (customerId) for aggregation.
count()	Count records per key → produces a KTable<key, Long>.
aggregate()	Custom accumulator — initializer + accumulator function per key.
selectKey()	Re-key the stream to a different field before aggregating.
Materialized.as()	Named state store — RocksDB by default, survives restarts.
@EnableKafkaStreams	Required annotation — activates Kafka Streams in Spring Boot.

6.2 Consumer Lag Monitoring

★ ADVANCED TOPIC: Consumer Lag: key operational metric — measures how far behind consumers are from the latest messages.

Consumer lag = endOffset – committedOffset per partition. A lag of 0 means fully caught up. Growing lag means the consumer cannot keep up with the producer rate. Implemented in payment-service, shipping-service, and dlt-handler-service via AdminClient.

```
// ConsumerLagService.java
// Step 1: Get committed offsets for this consumer group
Map committed =
```

```
    adminClient.listConsumerGroupOffsets(groupId)
               .partitionsToOffsetAndMetadata().get(10, SECONDS);

// Step 2: Get end offsets (latest available)
Map endOffsets = adminClient
    .listOffsets(partitions -> OffsetSpec.latest())
    .all().get(10, SECONDS);

// Step 3: Calculate lag
long lag = endOffset - committedOffset;  // 0 = healthy, >0 = lagging
```

■ Endpoint: GET /api/consumer-lag on ports 8082, 8083, 8085. Response includes totalLag, per-partition breakdown, and HEALTHY/LAGGING status.

6.3 Schema Registry + Avro — avro-service

★ ADVANCED TOPIC: Schema Registry: enterprise-grade schema governance. Avro: binary serialization for production throughput.

Avro replaces JSON for Kafka message serialization. Messages are binary (compact, fast). Schemas are centrally stored and versioned in Confluent Schema Registry. Schema evolution lets you add new optional fields without breaking existing consumers.

Aspect	JSON (order-service)	Avro (avro-service)
Format	Human-readable text	Binary bytes
Size	Large (fields repeated)	Small (schema shared)
Schema enforcement	None	Schema Registry
Schema evolution	Manual, error-prone	Built-in, backwards compatible
Serializer	JsonSerializer	KafkaAvroSerializer
Deserializer	JsonDeserializer	KafkaAvroDeserializer
Extra config	None	schema.registry.url required
Wire format	{ 'field': 'value' }	[magic][schemaId][binary]

```
# order-event.avsc (Avro Schema Definition)
{
  "type": "record", "name": "OrderEvent",
  "namespace": "com.mateen.avro",
  "fields": [
    {"name": "eventId",      "type": "string"},
    {"name": "eventType",    "type": {"type": "enum", "symbols": ["ORDER_PLACED", ...]}}
,
    {"name": "customerId",   "type": "string"},
    {"name": "message",      "type": ["null", "string"], "default": null}  ← nullable
  ]
}

# avro-maven-plugin generates OrderEvent.java at compile time:
# target/generated-sources/avro/com/mateen/avro/OrderEvent.java
```

7. Phase 3 — Production Kafka (Low-Level Design)

★ ADVANCED TOPIC: Phase 3 covers production-readiness features: exactly-once delivery, DB integration, and security.

7.1 Kafka Transactions — order-service

★ ADVANCED TOPIC: Kafka Transactions: atomic writes to multiple topics — either ALL succeed or NONE are visible.

A transactional producer sends messages to two topics atomically. If the second write fails, the first is also rolled back. Consumers configured with isolation.level=read_committed only see messages from committed transactions — never from aborted or in-flight ones.

```
# application.yml — order-service producer
spring.kafka.producer:
  transaction-id-prefix: order-tx-  # enables transactional producer

# application.yml — payment-service consumer
spring.kafka.consumer.properties:
  isolation.level: read_committed   # only read committed transactions

// Java — TransactionalOrderService.java
kafkaTemplate.executeInTransaction(ops -> {
    ops.send(orderEventsTopic, customerId, placedEvent);    // topic 1
    ops.send(orderConfirmedTopic, customerId, confirmedEvent); // topic 2
    return true;
    // If ANY send throws → BOTH are automatically rolled back
});
```

Concept	Description
transaction-id-prefix	Enables transactional mode. Spring appends instance counter: order-tx-0.
executeInTransaction()	Wraps sends in begin/commit/abort. Atomic — all or nothing.
commitTransaction()	Both messages become visible simultaneously to read_committed consumers.
abortTransaction()	Both messages permanently skipped — as if they never existed.
read_uncommitted (default)	Consumer sees ALL messages including in-flight/aborted transactions.
read_committed	Consumer ONLY sees messages from fully committed transactions.
Idempotence vs Transactions	Idempotence: no duplicates on single topic. Transactions: atomic multi-topic.

7.2 Kafka Connect — JDBC Sink Connector

★ ADVANCED TOPIC: Kafka Connect: framework for moving data between Kafka and external systems without application code.

Kafka Connect reads the order-events topic and writes each message to a PostgreSQL table automatically. No Java code needed — only a JSON connector configuration. The table is created automatically from the message schema. Upsert mode prevents duplicates.

```
// connectors/order-events-sink.json
{
  "name": "order-events-jdbc-sink",
  "config": {
    "connector.class": "io.confluent.connect.jdbc.JdbcSinkConnector",
    "topics": "order-events",
```

```
    "connection.url": "jdbc:postgresql://postgres:5432/paymentdb",
    "auto.create": "true",      // creates table on first message
    "auto.evolve": "true",      // adds columns for new fields
    "insert.mode": "upsert",    // no duplicates on orderId
    "pk.mode": "record_value",
    "pk.fields": "orderId",
    "table.name.format": "order_events_sink",
    "transforms": "flatten",    // nested JSON → flat columns
    "transforms.flatten.delimiter": "_"
  }
}
```

Feature	Description
Source Connector	External system → Kafka (e.g. CDC: database changes → Kafka)
Sink Connector (this)	Kafka → External system (e.g. order-events → PostgreSQL table)
auto.create: true	Table created automatically from first message's schema
auto.evolve: true	New fields in messages → new columns added to table
insert.mode: upsert	Same orderId → row updated, not duplicated
Flatten transform	Nested JSON {order:{orderId}} → flat column order_orderId
connect-offsets topic	Fault tolerance: stores read offsets. Restart = resume from last offset.
REST API (port 8083)	GET /connectors, POST /connectors, GET /connectors/{name}/status

7.3 SSL/SASL Security

★ ADVANCED TOPIC: SSL/SASL: production security layer — all Kafka connections require authentication.

SASL/PLAIN authenticates clients with username/password. Each microservice has its own credential. The broker validates credentials against kafka_server_jaas.conf. In production, combine with SSL encryption (SASL_SSL) and use SCRAM-SHA-512 instead of PLAIN.

```
# kafka_server_jaas.conf (mounted into Kafka container)
KafkaServer {
    org.apache.kafka.common.security.plain.PlainLoginModule required
    username="admin" password="admin-secret"
    user_order-service="order-secret"
    user_payment-service="payment-secret"
    user_shipping-service="shipping-secret";
};

# application.yml — any service connecting to secured broker
spring.kafka.producer.properties:
  security.protocol: SASL_PLAINTEXT
  sasl.mechanism: PLAIN
  sasl.jaas.config: >
    org.apache.kafka.common.security.plain.PlainLoginModule required
    username="order-service" password="order-secret";
```

Mechanism	How	Production Ready
PLAIN	Username/password in cleartext	Only with SSL encryption

SCRAM-SHA-256	Challenge/response hashing	Yes
SCRAM-SHA-512	Stronger hashing	Yes (recommended)
GSSAPI (Kerberos)	Kerberos tickets	Enterprise
OAUTHBEARER	OAuth 2.0 tokens	Cloud-native

8. Infrastructure & Docker Compose

All infrastructure runs via Docker Compose. The docker-compose.yml in kafka-advanced/ starts all required services.

Service	Image	Port	Purpose
Kafka	confluentinc/cp-kafka:7.6.0	9092	Message broker
Zookeeper	confluentinc/cp-zookeeper:7.6.0	2181	Kafka coordinator
Schema Registry	cp-schema-registry:7.6.0	8091	Avro schema storage
Kafka UI	provectuslabs/kafka-ui:latest	8080	Topics/schemas browser
Kafka Connect	cp-kafka-connect:7.6.0 (custom)	8083	JDBC Sink Connector runtime
PostgreSQL	postgres:15-alpine	5432	Payment records + sink table
Elasticsearch	elasticsearch:8.15.3	9200	Log storage
Logstash	logstash:8.15.3	5000	Log pipeline (TCP JSON → ES)
Kibana	kibana:8.15.3	5601	Log dashboard
Zipkin	openzipkin/zipkin:3.4	9411	Distributed tracing

```
# Start all infrastructure
cd kafka-advanced/
docker compose up -d
```

9. Port Reference

Service	Port	Type
order-service	8081	Spring Boot (REST)
payment-service	8082	Spring Boot (REST + Consumer)
shipping-service	8083	Spring Boot (Consumer)
audit-service	8084	Spring Boot (Kafka Streams)
dlt-handler-service	8085	Spring Boot (Consumer)
avro-service	8086	Spring Boot (Producer + Consumer)
Kafka Broker	9092	Kafka (PLAINTEXT)
Zookeeper	2181	Zookeeper
Schema Registry	8091	HTTP REST API
Kafka UI	8080	HTTP Dashboard
Kafka Connect	8083	HTTP REST API (Docker)
PostgreSQL	5432	PostgreSQL
Zipkin	9411	HTTP Dashboard
Elasticsearch	9200	HTTP REST API
Kibana	5601	HTTP Dashboard
Logstash	5000	TCP (JSON)

10. How to Run — Step by Step

Prerequisites

- Java 17 installed (java -version should show 17.x)
- Maven 3.x installed (mvn -version)
- Docker Desktop running
- Git (project cloned from github.com/mateenms/kafka)

Step 1 — Start Infrastructure

```
cd /path/to/kafka/kafka-advanced
docker compose up -d

# Verify Kafka is up:
curl http://localhost:8080          # Kafka UI
curl http://localhost:8091/subjects # Schema Registry
```

Step 2 — Build All Services

```
# Build shared library first
mvn clean install -q -f base-domain/pom.xml

# Build all services
for svc in order-service payment-service shipping-service \
           dlt-handler-service audit-service avro-service; do
  mvn clean install -DskipTests -q -f $svc/pom.xml && echo "OK: $svc"
done
```

Step 3 — Start Services (6 terminals)

```
java -jar order-service/target/order-service-1.0.0.jar          # port 8081
java -jar payment-service/target/payment-service-1.0.0.jar      # port 8082
java -jar shipping-service/target/shipping-service-1.0.0.jar    # port 8083
java -jar audit-service/target/audit-service-1.0.0.jar          # port 8084
java -jar dlt-handler-service/target/dlt-handler-service-1.0.0.jar  # port 8085
java -jar avro-service/target/avro-service-1.0.0.jar            # port 8086
```

Step 4 — Register Kafka Connect Sink Connector

```
bash kafka-connect/register-connector.sh

# Verify:
curl http://localhost:8083/connectors/order-events-jdbc-sink/status
```

11. Test Scenarios

Normal Order Flow

```
curl -X POST http://localhost:8081/api/v1/orders \
  -H "Content-Type: application/json" \
  -d '{
    "customerId": "CUST-001",
    "productId": "PROD-101",
    "productName": "iPhone 15",
    "quantity": 2,
    "price": 999.99
  }'

# Expected: payment processed → saved to DB → shipment scheduled
```

Retry + DLT Demo (price=1 triggers failure)

```
curl -X POST http://localhost:8081/api/v1/orders \
  -H "Content-Type: application/json" \
  -d '{"customerId":"CUST-002","productName":"Fail Test","quantity":1,"price":1}'

# Expected: payment-service retries 3x (1s → 2s → 4s)
# → message goes to order-events-dlt
# → dlt-handler-service logs ERROR
```

Kafka Transactions Demo

```
curl -X POST http://localhost:8081/api/v1/orders/transactional \
  -H "Content-Type: application/json" \
  -d '{"customerId":"CUST-001","productName":"Laptop","quantity":1,"price":1299.99}'

# Expected: both order-events AND order-confirmed-events receive messages atomically
# Verify in Kafka UI → both topics show the message at the same time
```

Avro + Schema Registry Demo

```
curl -X POST http://localhost:8086/api/avro/orders \
  -H "Content-Type: application/json" \
  -d '{"customerId":"CUST-001","productName":"Laptop","quantity":1,"price":"1299.99"}'

# Verify schema registered:
curl http://localhost:8091/subjects
# Returns: ["avro-order-events-value"]
```

Consumer Lag Check

```
curl http://localhost:8082/api/consumer-lag   # payment-service
curl http://localhost:8083/api/consumer-lag   # shipping-service
curl http://localhost:8085/api/consumer-lag   # dlt-handler-service

# Response: {"groupId":"...","status":"HEALTHY","totalLag":0,...}
```

Kafka Connect — Verify PostgreSQL Sink

```
# After sending an order, check PostgreSQL:
docker exec -it postgres-payment psql -U postgres -d paymentdb \
  -c 'SELECT * FROM order_events_sink;'

# Each order-events message should appear as a row automatically
```

12. Configuration Reference

Key Kafka Producer Properties

Property	Value	Purpose
enable.idempotence	true	No duplicate messages on retry
acks	all	All replicas confirm before ACK (required for idempotence)
retries	3	Retry 3 times on send failure
max.in.flight.requests.per.connection	5	Max unacked requests (≤5 with idempotence)
transaction-id-prefix	order-tx-	Enables transactional producer
key-serializer	StringSerializer	Partition key serialization
value-serializer	JsonSerializer / KafkaAvroSerializer	Message value serialization

Key Kafka Consumer Properties

Property	Value	Purpose
group-id	service-name-group	Consumer group — messages balanced across instances
auto-offset-reset	earliest	Start from beginning if no offset committed
isolation.level	read_committed	Only see committed transaction messages (Phase 3)
SPECIFIC_AVRO_READER	true	Return typed Java class from Avro (not GenericRecord)
spring.json.trusted.packages	*	Allow deserialization of base-domain DTOs

Advanced Topics Summary

★ ADVANCED TOPIC: The following are the most advanced concepts in this project — study these carefully.

★ Concept	Service	Why Advanced	Key Config/Code
Kafka Streams	audit-service	Stateful processing, state stores, KStream/KTable DSL	@EnableKafkaStreams, KafkaStreamsConfiguration b
Schema Registry + Avro	avro-service	Binary format, schema versioning, schema evolution	KafkaAvroSerializer, SPECIFIC_AVRO_READER=tru
Kafka Transactions	order-service	Exactly-once across multiple topics, consumer isolation	transaction-id-prefix, executeInTransaction(), read_co
Kafka Connect	kafka-connect	No-code data pipeline, fault-tolerant, standalone	JDBC Sink, auto.create, upsert mode
Consumer Lag Monitoring	payment/shipping/dlt	Production observability, AdminClient API	AdminClient, listConsumerGroupOffsets()
SASL/PLAIN Security	security-config	Broker authentication, per-service credentials	SASL_PLAINTEXT, PlainLoginModule, JAAS config